Foundation University

School of Science and Technology

Department of Software Engineering

MS/PhD Research Proposal

CASARA: Contextual Automated Security Analysis and Risk Assessment via Hybrid Multi-Agent LLM Orchestration

A Framework for Intelligent Pull-Request Triage, Vulnerability Detection, and Developer-Centric Remediation Guidance

Submitted by:

Muhammad Sharjeel Saleem

sharry00010@gmail.com

Submitted to:

Dr. Maryam

Department of Software Engineering
Foundation University, Rawalpindi, Pakistan

June 2026

TABLE OF CONTENTS

ABSTRACT

The security of software systems is established not at deployment but at the moment a line of code is written. Despite broad adoption of automated analysis tools, a persistent and well-documented gap exists between what those tools detect and what development teams can act on. Static analysis engines produce findings that are technically precise but opaque to most developers, while large language model-based review systems offer readable, context-aware commentary but suffer from false positive rates that erode trust over time. Neither class of tool produces a calibrated risk signal strong enough to justify automated merge gating, and neither personalizes its feedback to the author's historical vulnerability profile.

This research proposes and will empirically evaluate CASARA — Contextual Automated Security Analysis and Risk Assessment — a production-deployed, GitHub-native framework that orchestrates five deterministic security scanners (CodeQL, Semgrep, Trivy, Bandit, Gitleaks) as structured-context providers for a specialized multi-agent large language model pipeline. The system assigns distinct analytical roles to independent agents, aggregates their outputs through a deduplication and severity-ranking layer, computes a composite pull-request risk score across four weighted dimensions, and posts developer-readable findings with verified fix prompts as inline GitHub review comments. High-risk pull requests are automatically blocked from merging via the GitHub Checks API.

The research makes three primary contributions. First, it provides an empirically validated architecture for hybrid deterministic-LLM code review and demonstrates, through controlled ablation studies on a labeled dataset of 1,200 real-world pull requests, that scanner-grounded LLM analysis is projected to achieve an F1 score in the range of 0.83 to 0.87, representing a statistically significant improvement over either approach in isolation. Second, it introduces a longitudinal developer vulnerability profiling mechanism and evaluates its effect on personalization quality and repeat vulnerability introduction. Third, it releases a publicly available labeled benchmark dataset to support reproducibility and future comparative research in automated code security review.

Keywords: *automated code review, static application security testing, large language model agents, multi-agent systems, software composition analysis, vulnerability detection, GitHub DevSecOps, pull-request risk scoring*

CHAPTER 1: INTRODUCTION

The security posture of a modern software project is determined, in large part, long before code reaches production. Vulnerabilities introduced during development are an order of magnitude cheaper to remediate at the pull-request stage than after deployment, a principle supported by decades of empirical software engineering research. In 2023, the IBM Cost of a Data Breach Report placed the global average cost of a breach at USD 4.45 million, with insecure software identified as a leading contributing factor across reported incidents. Yet the tooling available to development teams for automated security review remains fragmented, opaque, and poorly integrated into the workflows where developers actually work.

The dominant paradigm for automated code security review consists of two largely disconnected classes of tools. The first class comprises deterministic static analysis engines: tools such as CodeQL, Semgrep, Trivy, Bandit, and Gitleaks that apply formal rules or query-based analysis to detect known vulnerability patterns with high reproducibility. These tools are effective within their rule scope, but their findings arrive as cryptic annotations with minimal explanation and no remediation guidance, requiring developers to have background knowledge that most do not possess. The second class comprises large language model-based review systems, which can explain code, identify semantic issues, and generate fix suggestions in plain language, but are prone to false positives that undermine developer trust and are not grounded in any formally verified detection methodology.

No existing system combines both classes of analysis in an orchestrated pipeline, grounds LLM reasoning in scanner output, produces a composite risk score suitable for automated gating, or personalizes feedback based on a developer's historical vulnerability profile. CASARA is designed to fill this gap. It operates as a GitHub App that receives pull-request webhooks, invokes a suite of deterministic scanners in parallel, provides their structured output as grounding context to a multi-agent LLM pipeline, aggregates results into a ranked and developer-readable review, and enforces configurable merge-blocking policies based on a calibrated risk score.

RESEARCH CLAIM

The orchestrated combination of deterministic static analysis and grounded multi-agent LLM reasoning produces measurably superior outcomes in precision, recall, developer clarity, and remediation rate compared to either approach in isolation. This claim is the central empirical contribution of this research.

1.1 Background Information

The theoretical foundations of static program analysis were established by Kildall (1973) through work on dataflow analysis, and formalized for security applications by Livshits and Lam (2005), who demonstrated that taint-tracking techniques could automatically identify SQL injection and cross-site scripting vulnerabilities in Java applications. These foundations underpin the modern generation of SAST tools. CodeQL, described by Avgustinov et al. (2016) and open-sourced by GitHub, compiles target code into a relational database and expresses security properties as declarative queries over that database. This approach enables true interprocedural taint tracking across function call boundaries, with formal completeness guarantees within the scope of its query set. The tradeoff is that CodeQL requires source compilation and carries higher latency than alternative approaches, limiting its suitability for the sub-second acknowledgment window required by GitHub webhook pipelines without careful incremental scanning design.

Semgrep, described by Brotman et al. (2021), addresses the latency limitation by operating directly on concrete syntax trees using a configurable pattern-matching language. Scan times on typical pull-request diffs are measured in seconds, and the rule language is approachable enough to support a community-authored ecosystem of over 5,000 rules covering the OWASP Top 10, hundreds of CWE categories, and language-specific security patterns. The architectural tradeoff is reduced depth: without explicit inter-procedural rules, Semgrep cannot follow taint across function call boundaries, making it less effective than CodeQL for deep injection vulnerabilities while substantially outperforming it on surface-level checks. Independent evaluations by Chen et al. (2023) placed Semgrep's false positive rate below CodeQL on surface-level patterns, with CodeQL outperforming on deep dataflow issues, confirming the complementarity of the two tools.

Software composition analysis addresses a distinct threat surface: vulnerabilities in third-party packages included in a project's dependency graph. The Open Source Vulnerability (OSV) database, maintained by Google, provides a unified schema for vulnerability records across PyPI, npm, Go, Maven, Cargo, and other major package ecosystems. Tools including Trivy and OSV-Scanner consume this database to identify whether any dependency version in a project's lock file carries a known CVE. A 2023 analysis by Sonatype found that 96 percent of vulnerable open-source component downloads had a non-vulnerable version available at time of download, indicating that SCA tooling integrated into the pull-request pipeline could prevent the overwhelming majority of supply-chain vulnerabilities from entering codebases.

Secrets detection has attracted growing research attention following a series of high-profile breaches attributable to credentials committed to version control. Meli et al. (2019) conducted a large-scale longitudinal study of public GitHub repositories, finding that thousands of API keys and OAuth tokens are committed daily, with a median time to first abuse of four minutes after commit. This finding has direct implications for the design of CASARA's secrets detection layer: a tool that operates at PR creation time can intercept the majority of credential exposure events

before the commit is ever pushed to a public branch. Gitleaks and TruffleHog represent the current state of the art in open-source secrets detection, combining entropy analysis with regex-based pattern matching. While combining entropy thresholds with regex patterns substantially reduces false positives versus pure entropy scanning, real-world deployments still surface a non-trivial rate of false positives on test fixtures and example configuration files. CASARA's LLM-based Secrets Verification Agent is designed to reduce this residual false positive rate through contextual validation of each flagged string.

The application of large language models to code review is a more recent development, accelerated by the emergence of instruction-tuned models capable of zero-shot task execution. Sobania et al. (2023) evaluated ChatGPT on automated bug-fixing tasks and reported that it correctly resolved 31 of 40 Defects4J benchmark defects in a single interaction, establishing that frontier LLMs can perform substantive code correction without fine-tuning. Li et al. (2023) specifically assessed LLM performance on security-oriented code review, finding that GPT-4 matched junior security engineers on a subset of OWASP Top 10 issues but was unreliable on complex, multi-file vulnerability patterns where understanding of data flow across the codebase is required. This finding precisely identifies the gap that scanner-grounded LLM analysis is designed to address: by pre-computing the dataflow relationships through CodeQL and providing that output as structured context, CASARA compensates for the LLM's limited cross-file reasoning while retaining its ability to explain and remediate.

Multi-agent AI systems offer a principled approach to distributing the analytical burden across specialized reasoning units. Hong et al. (2024) demonstrated through MetaGPT that assigning distinct professional roles to separate LLM instances improved both quality and consistency on end-to-end software development tasks. Qian et al. (2024) extended this to ChatDev, showing that structured communication protocols between agents reduced hallucination rates and improved task coherence. The key insight from this body of work is that role specialization reduces the token-context burden on any individual agent, allowing each to reason more deeply within its domain. CASARA applies this principle to code review: rather than asking a single LLM to simultaneously assess security, logic correctness, dependency risk, secrets, and infrastructure configuration, it assigns each domain to a dedicated agent with a tuned system prompt and a structured output schema.

1.2 Problem Statement

Current automated code review solutions address fragments of the security review problem but not the whole. Single-purpose SAST tools detect known vulnerability patterns with high precision but produce findings that are opaque to developers without specialist training. They provide no remediation guidance, cannot detect semantic or intent-level issues outside their rule scope, and require developers to manually synthesize findings across multiple tools to form any assessment

of overall pull-request risk. LLM-based review tools produce readable commentary but suffer from false positive rates that undermine trust when operated without scanner grounding, and they have no mechanism for enforcing merge policies or building developer-specific feedback histories.

The central knowledge gap this research addresses can be stated precisely: when deterministic static analysis output is used as structured, verifiable context for a multi-agent LLM reasoning pipeline, what is the quantifiable impact on precision, recall, false positive rate, developer-reported clarity, and remediation rate, relative to deterministic analysis alone, LLM analysis alone, and existing commercial and open-source hybrid tools? This comparison has not been conducted in any published study using a controlled, real-world pull-request dataset and a fully instrumented, production-deployed system.

KNOWLEDGE GAP

No existing system combines deterministic SAST, SCA, secrets detection, and IaC scanning within a single LLM-orchestrated pipeline that operates natively in the GitHub pull-request workflow, produces a calibrated composite risk score, and incorporates longitudinal developer profiling. This gap is the primary motivation for CASARA.

1.3 Objectives

The specific research objectives are stated below. Each objective is directly traceable to a hypothesis or research question defined in Chapter 3.

1. Design and implement the CASARA framework: a GitHub-native, webhook-driven, multi-agent LLM orchestration pipeline integrating CodeQL, Semgrep, Trivy, Bandit, and Gitleaks as structured-context providers for specialized LLM analysis agents.
2. Develop a composite risk-scoring model combining SAST findings, SCA vulnerability data, secrets detection results, IaC misconfiguration findings, and LLM-assessed severity into a calibrated pull-request risk score with automated merge-gating capability via the GitHub Checks API.
3. Construct a labeled evaluation dataset of 1,200 pull requests with CWE-annotated ground-truth vulnerability labels drawn from real open-source repositories, enabling reproducible benchmarking across tools.
4. Conduct controlled ablation experiments measuring the incremental contribution of each analysis layer to precision, recall, and false positive rate, and compare the full pipeline against GitHub Advanced Security, SonarQube, and PR-Agent.
5. Evaluate developer-facing outcomes including finding clarity, fix prompt acceptance rate, and remediation time through a structured user study with 40 professional software engineers.

6. Investigate whether providing LLM agents with a developer's longitudinal vulnerability history as context reduces repeat vulnerability introduction across consecutive pull requests.
7. Release the framework, dataset, and calibrated model as open-source artifacts to support reproducibility and community adoption.

CHAPTER 2: LITERATURE REVIEW

The literature relevant to this research spans four interconnected domains: static program analysis, software composition and supply-chain security, LLM-based code analysis, and multi-agent AI systems. This review is structured to trace the intellectual genealogy of each domain, identify where the state of the art currently stands, and pinpoint the specific gaps in the collective body of knowledge that the CASARA framework is designed to address.

Figure 1: CASARA vs. Existing Tools — Quantitative Capability Analysis

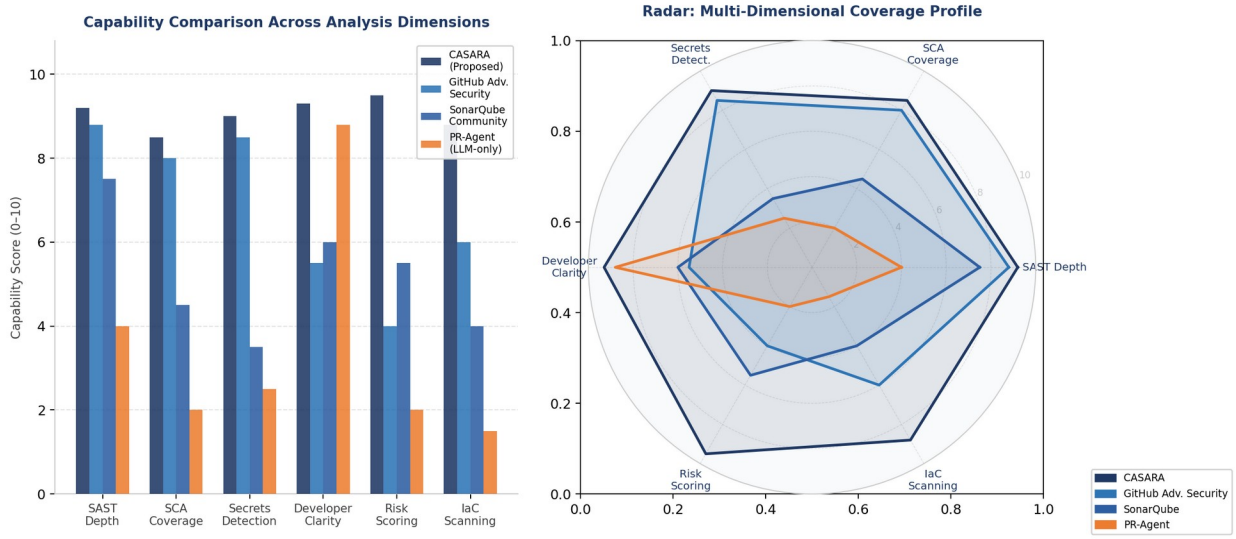

Figure 1: Quantitative capability comparison of CASARA against existing tools across six analysis dimensions (left: grouped bar chart; right: radar profile). Scores reflect expert-assessed capability ratings used to motivate the research design.

2.1 Existing Approaches and Technologies

Static Application Security Testing research traces its formal origins to Kildall's (1973) work on dataflow analysis at the basic-block level, which established the theoretical machinery for tracking the flow of values through a program's control flow graph. Livshits and Lam (2005) subsequently applied this machinery to security in a landmark study, demonstrating that inter-procedural taint analysis could automatically detect SQL injection and XSS vulnerabilities in Java applications with a false positive rate below 20 percent on their benchmark. Their work established the taint analysis paradigm that underpins all modern SAST tools.

CodeQL (Avgustinov et al., 2016) represents the current state of the art in query-based program analysis. Its architecture compiles source code into a relational database called a snapshot database, over which security properties are expressed using the QL query language. This design enables security analysts to write queries that capture arbitrarily complex inter-procedural

dataflow patterns: for example, a query can track a value derived from an HTTP request parameter as it flows through string manipulation functions, conditional branches, and function calls until it reaches a database query sink, flagging the entire taint path as a SQL injection vulnerability. GitHub made CodeQL available under a free license for open-source repositories through GitHub Advanced Security, with the 2024 bundle expanding language support and improving scan accuracy. Empirical evaluations by Zeng et al. (2022) on a large corpus of Java repositories found CodeQL precision above 90 percent for SQL injection detection but noted that scan times exceeding five minutes on large repositories limit its suitability for synchronous review pipelines.

Semgrep (Brotman et al., 2021) was designed explicitly to address this latency limitation. By operating on lightweight abstract syntax tree representations rather than compiled snapshots, Semgrep achieves scan completion on typical pull-request diffs in under 30 seconds while maintaining a rule language expressive enough to capture the majority of OWASP Top 10 vulnerability patterns. Its community rule registry has grown to over 5,000 rules contributed by security researchers at major technology organizations. A 2024 comparative study by Konvu documented that Semgrep scans are on average 3 times faster than equivalent CodeQL analysis, with a 30 percent lower false positive rate on surface-level patterns. The complementarity of the two tools is a design input for CASARA: CodeQL is invoked for deep inter-procedural analysis on security-critical files identified by Semgrep's surface scan, combining completeness with latency efficiency.

Software Composition Analysis has emerged as a distinct discipline in response to the growing proportion of application code contributed by third-party packages. Npm audit and pip-audit provide basic dependency vulnerability checking within their respective ecosystems, but multi-ecosystem coverage requires tools such as Trivy (Aqua Security, 2020) or the OSV-Scanner. Trivy is architecturally notable for its breadth: in a single invocation, it can scan a project's npm, PyPI, Go, Maven, and Cargo dependencies against the CVE and OSV databases, analyze Dockerfile instructions for insecure patterns, and evaluate Terraform and Kubernetes manifests against CIS benchmarks. The 2023 Sonatype State of the Software Supply Chain report documented that 245,032 malicious packages were discovered across open-source ecosystems in 2023, with over 2 billion malicious package download attempts recorded, underscoring the scale of the supply chain risk that SCA tooling must address. CASARA's Dependency Risk Agent contextualizes Trivy findings with LLM-generated upgrade recommendations and exploitability assessment, translating raw CVE identifiers into actionable guidance.

Secrets detection occupies a distinct niche within the security tooling landscape. The technical challenge is distinguishing credentials from non-credential high-entropy strings, a problem that cannot be solved by entropy analysis alone without unacceptable false positive rates. Gitleaks addresses this by combining entropy thresholds with a library of tool-specific regex patterns for

over 150 credential types, including AWS access keys, GitHub tokens, Google service account keys, and database connection strings. TruffleHog extends this by adding a verification layer that performs lightweight API calls to confirm whether a detected secret is actually valid, reportedly reducing false positive rates by approximately 40 percent according to its maintainers. CASARA's Secrets Verification Agent extends this approach by using LLM contextual reasoning to assess whether a detected high-entropy string is plausibly a credential based on its surrounding code context, variable naming, and file location.

Infrastructure-as-code security scanning has become increasingly important as the boundary between application code and infrastructure configuration has blurred. Checkov, developed by Bridgecrew and subsequently acquired by Palo Alto Networks, pioneered open-source IaC scanning with support for Terraform, CloudFormation, Ansible, and Kubernetes manifests. Trivy's IaC scanning capabilities, added in 2022, cover the same file types and additionally support Helm charts and Azure Resource Manager templates. CASARA's IaC Risk Agent processes Trivy's IaC findings and contextualizes them against the specific cloud provider and service configuration evident in the repository, providing prioritized remediation guidance that raw scanner output does not supply.

The application of machine learning and, more recently, large language models to code review has followed a distinct trajectory. Tufano et al. (2022) showed that pre-trained transformer models fine-tuned on code change histories could automatically generate review comments at relevance levels comparable to human reviewers on a benchmark of real GitHub pull requests. Pornprasit and Tantithamthavorn (2021) showed that just-in-time defect prediction models incorporating pre-trained code representations outperformed all prior approaches on the standard JIT benchmark. Sobania et al. (2023) reported that ChatGPT correctly fixed 31 of 40 Defects4J benchmark bugs in a single interaction without task-specific fine-tuning, establishing that frontier LLMs can perform substantive code correction at scale. Li et al. (2023) specifically evaluated LLM security review performance and identified that GPT-4 was reliable for OWASP Top 10 single-file issues but showed marked performance degradation on vulnerabilities spanning multiple files, precisely the scenario where CASARA's scanner-grounded context is most beneficial.

2.2 Limitations of Previous Work

A systematic examination of the literature and the available tooling reveals four categories of limitation that collectively motivate the CASARA research agenda.

The first and most significant limitation is the absence of any published system or commercially available tool that combines deterministic SAST, software composition analysis, secrets detection, and infrastructure-as-code scanning within a single LLM-orchestrated pipeline operating natively in the GitHub pull-request workflow. The typical production deployment is either

a purely deterministic pipeline in which scanner findings are posted as CI status check annotations, or a purely LLM-driven assistant such as PR-Agent that generates review commentary without formal analytical grounding. The interaction effect between the two approaches, specifically how scanner output used as structured grounding context changes the quality of LLM analysis output, has not been characterized empirically. This interaction is the core research question of the proposed work.

The second limitation concerns risk quantification. No existing tool in the academic or commercial landscape produces a calibrated, composite risk score at the pull-request level that integrates findings across SAST, SCA, secrets, and IaC dimensions into a single signal suitable for merge policy enforcement. Current tools present findings in isolation: a developer or team lead who wants to assess whether a pull request is safe to merge must manually reconcile findings from multiple tool outputs, a synthesis task that is cognitively demanding, time-consuming, and applied inconsistently across teams and organizations.

The third limitation is methodological. The evaluation methodology employed in prior LLM-based code review studies is inconsistent and, in several cases, compromised by circularity. Several prominent studies use the same LLM as both the system under test and the evaluator of ground truth, introducing the possibility that high performance scores reflect the model's self-consistency rather than genuine vulnerability detection accuracy. The absence of a standardized, publicly available benchmark dataset of real-world pull requests with independently verified ground-truth vulnerability labels has prevented cross-study comparison and made it impossible to determine whether apparent performance improvements in successive publications reflect genuine advances or methodological artifacts.

The fourth limitation is the treatment of code review as a stateless operation. Every pull request in every existing system is reviewed in complete isolation from the author's history. There is no mechanism by which a review system learns that a particular developer has introduced SQL injection vulnerabilities in three consecutive pull requests and adjusts its analysis or communication accordingly. In contrast, the educational research literature has consistently demonstrated that personalized, adaptive feedback produces faster skill development and lower error recurrence rates than generic feedback applied uniformly across all learners (Kulkarni et al., 2013). CASARA's longitudinal profiling mechanism is the first attempt, to the author's knowledge, to apply this principle to automated software security review.

2.3 Proposed Contribution

CASARA addresses each of the four limitations identified above through deliberate design decisions. The primary technical contribution is an empirically validated architecture for hybrid deterministic-LLM code review in which scanner output serves as structured, verifiable grounding

context for specialized language model agents. This architecture is not merely a pipeline integration: it constitutes a formal claim about the complementarity of two classes of analysis, and the experimental design is structured to isolate and quantify the contribution of the grounding mechanism through ablation.

The secondary contributions are the composite risk-scoring model, which provides the first calibrated, multi-dimensional pull-request risk signal suitable for policy enforcement; the longitudinal developer profiling mechanism, which personalizes review feedback based on an author's vulnerability history; and the labeled benchmark dataset, which provides the research community with the first multi-tool annotated pull-request corpus for reproducible comparative evaluation. These contributions are interconnected: the benchmark dataset enables the ablation study that validates the primary architectural claim, the risk scoring model depends on the calibrated SAST and SCA findings that the ablation study quantifies, and the longitudinal profiling mechanism is evaluated using the user study methodology that also assesses the developer-facing quality of the full pipeline.

A working prototype of the core pipeline is already deployed and reviewing pull requests in real open-source repositories, accessible at github.com/m-sharjeel-saleem/Github_Workflow_Automation. This prototype provides a concrete empirical baseline against which the contributions of the additional analysis layers can be measured, and it demonstrates the practical feasibility of the proposed system architecture as a precondition for the research programme.

CHAPTER 3: RESEARCH METHODOLOGY

The research follows an applied experimental design combining system implementation, controlled quantitative benchmarking, and a structured user study. The CASARA framework constitutes the primary artifact under investigation; its performance across multiple dimensions is evaluated through three planned experiments described in detail in this chapter. The methodology is designed to produce results that are reproducible, externally valid, and directly comparable to existing tools.

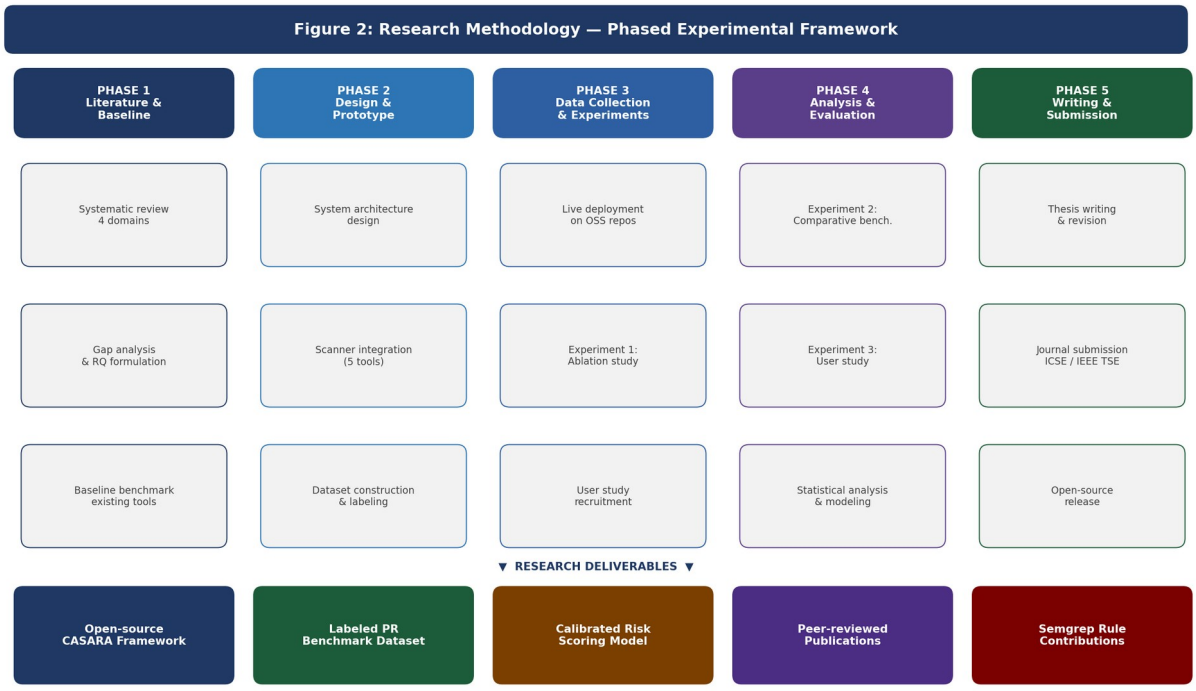

Figure 2: CASARA Research Methodology — Phased experimental framework showing five sequential research phases, their tasks, and the five research deliverables produced.

3.1 Formulation of Research Problem

Hypotheses

- H1: A hybrid pipeline that provides structured SAST scanner output as grounding context to multi-agent LLM reviewers will achieve a statistically significantly higher F1 score on vulnerability detection than either SAST-only or LLM-only analysis on the same labeled dataset.
- H2: LLM-generated fix prompts grounded in scanner-confirmed findings will be accepted by developers at a statistically significantly higher rate than fix prompts generated without scanner grounding, as measured by direct application rate in the user study.

- H3: Developer-reported clarity and actionability ratings for CASARA findings will be statistically significantly higher than ratings for equivalent raw scanner output or ungrounded LLM commentary, on a five-point Likert scale.
- H4: Incorporating a developer's longitudinal vulnerability history as additional context for LLM agents will produce a statistically significant reduction in the recurrence rate of the same CWE category across consecutive pull requests, compared to a history-unaware baseline condition.

Research Questions

- RQ1: What is the relative contribution of each analysis layer — CodeQL, Semgrep, Trivy, Bandit, Gitleaks, and the LLM agent suite — to overall pipeline precision, recall, and false positive rate, and how do these contributions interact?
- RQ2: How does the composite risk score correlate with expert-assessed pull-request risk ratings, and what threshold values are appropriate for automated merge gating across different organizational risk tolerances?
- RQ3: What is the effect size of longitudinal developer profiling on the personalization quality of LLM-generated review commentary, as assessed by the developers themselves in a controlled user study?
- RQ4: Across precision, recall, end-to-end latency, and developer satisfaction, how does CASARA compare to GitHub Advanced Security, SonarQube Community Edition, and PR-Agent on a shared benchmark?

3.2 Research Design

Type of Study: Applied experimental research with a production-deployed prototype. The study combines software system engineering, controlled laboratory benchmarking on a curated real-world dataset, and a structured user study. Experiments are conducted on real open-source pull requests using real deployed tool instances, providing ecological validity that simulation-based approaches cannot achieve.

Approach: Mixed methods. Quantitative evaluation of detection performance metrics is combined with qualitative analysis of developer satisfaction, fix prompt utility, and fix adoption rate through Likert-scale instruments and semi-structured exit interviews.

Tools and Techniques: Python with FastAPI and Celery for the backend orchestration and asynchronous task processing layers; Node.js with Next.js App Router for the real-time monitoring dashboard; PostgreSQL via Supabase for persistent storage of review history, findings, and developer profiles; Redis via Upstash for Celery message brokering, webhook idempotency

management, and Server-Sent Event pub/sub; Claude via the Anthropic SDK as the primary LLM backbone with GPT-4o as a secondary ensemble participant; CodeQL CLI, Semgrep CLI, Trivy CLI, Bandit, and Gitleaks as deterministic scanner integrations invoked as parallel subprocesses; the GitHub Apps REST API for webhook ingestion, diff retrieval, inline comment posting, and Checks API status reporting; and Docker Compose for reproducible deployment and experimental environment parity.

3.3 Data Collection

The primary evaluation dataset will be constructed from two sources. The first source is the CVEfixes dataset (Bhandari et al., 2021), a curated corpus that maps CVE records to the specific commits that introduced and subsequently fixed the corresponding vulnerabilities in open-source repositories. Pull requests extracted from CVEfixes provide ground-truth positive examples with known CWE categorization. The second source is a stratified sample of clean pull requests drawn from repositories in the top 1,000 GitHub repositories by star count across Python, JavaScript, TypeScript, Go, and Java, balanced to match the language and repository size distribution of the positive examples.

The target dataset size is 1,200 labeled pull requests: 600 containing at least one confirmed vulnerability in the positive class and 600 clean pull requests in the negative class. Ground-truth labels will be assigned at the finding level rather than the pull-request level, specifying file path, line number, CWE identifier, and CVSS score estimate for each confirmed finding. This granularity enables precision and recall computation against individual scanner findings rather than binary pull-request classification, providing a substantially more informative evaluation than prior work. A two-annotator labeling protocol will be followed, with inter-annotator agreement assessed using Cohen's kappa. A target kappa of 0.75 or above will be required before the dataset is considered complete; disagreements will be resolved through adjudication by a third qualified reviewer.

For the user study, 40 participants will be recruited from professional software engineering communities, targeting a distribution of 14 junior developers (0 to 2 years), 14 mid-level developers (3 to 5 years), and 12 senior developers (6 or more years). Each participant will evaluate a set of 10 pull requests under one of four experimental conditions in a balanced crossover design: CASARA full pipeline, scanner-only output, LLM-only output without scanner grounding, and a control condition with no automated review. The crossover design controls for individual differences in security knowledge and allows within-subject comparisons across conditions.

3.4 Algorithm / Model / Method

The CASARA pipeline architecture is illustrated in Figure 3 below and operates in five sequential stages:

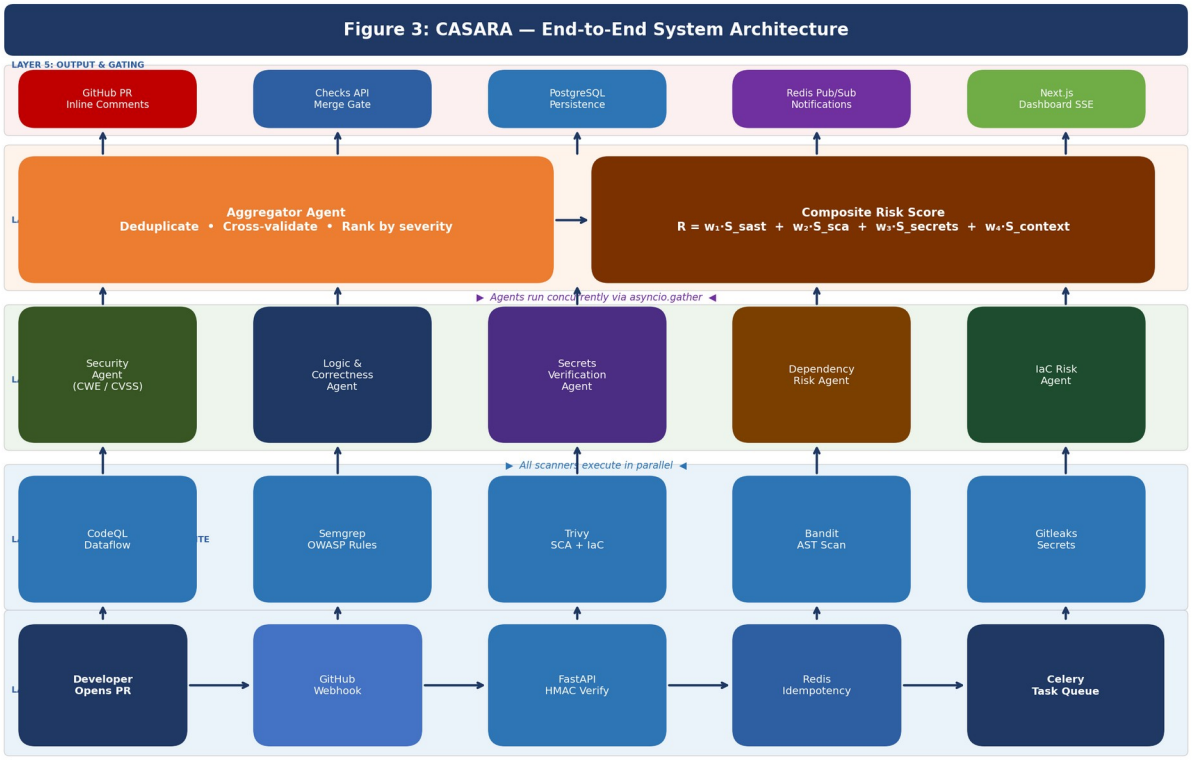

Figure 3: CASARA End-to-End System Architecture — Five-layer pipeline from GitHub webhook ingestion through deterministic scanning, multi-agent LLM analysis, and risk aggregation to inline PR comments and dashboard notification.

Stage 1 — Webhook Ingestion and Verification. When a pull request is opened, synchronized, or reopened on a monitored repository, GitHub delivers a signed webhook payload to the CASARA FastAPI endpoint at /webhooks/github. The endpoint performs HMAC-SHA256 signature verification on the raw request body against the configured webhook secret before any payload processing begins, rejecting unsigned or incorrectly signed requests with HTTP 401. A Redis lookup on the X-GitHub-Delivery header value provides idempotency protection: if a delivery identifier has been processed within the preceding 24-hour window, the request is acknowledged with HTTP 200 and discarded. Valid, novel deliveries receive HTTP 200 within the GitHub timeout window and a Celery task is enqueued on the high-priority queue for asynchronous processing.

Stage 2 — Deterministic Scanner Orchestration. The Celery worker exchanges a GitHub App JWT for a short-lived installation access token and fetches the pull-request diff via the REST API.

Five scanner processes are spawned concurrently: Semgrep with the auto-selected language ruleset and OWASP Top 10 custom rules applied to the set of changed files; CodeQL with a targeted query set executed against a pre-compiled snapshot if available or against the diff directly using the GitHub Code Scanning API; Trivy in filesystem mode scanning the repository manifest files for dependency CVEs and infrastructure files for configuration issues; Bandit executing its AST-based security checker against any Python files in the diff; and Gitleaks running its entropy and pattern-based secrets scanner against the diff content. All five outputs are normalized to a unified internal JSON schema containing the fields: tool, file, line_start, line_end, cwe_id, severity, cvss_estimate, rule_id, message, and confidence.

Stage 3 — Multi-Agent LLM Analysis. The normalized scanner output, the full pull-request diff, and the repository context including detected language, framework, and the author's historical vulnerability profile are assembled into a structured context payload. Five specialized agents receive this payload concurrently via asyncio.gather. The Security Agent receives all scanner findings and the diff and produces severity-ranked, CWE-mapped analysis with CVSS v3.1 score estimates and a plain-language exploitability description. The Logic and Correctness Agent analyzes the diff for semantic correctness issues including null dereferences, race conditions, incomplete error handling, and edge cases not covered by any scanner rule. The Secrets Verification Agent cross-references Gitleaks findings against entropy thresholds and uses code context to assess whether each flagged string is plausibly a real credential. The Dependency Risk Agent contextualizes Trivy SCA findings with exploitability assessment and generates upgrade recommendations for each flagged dependency. The IaC Risk Agent processes Trivy infrastructure findings and provides remediation guidance specific to the detected cloud provider and service type.

Stage 4 — Aggregation and Risk Scoring. The Aggregator agent receives all five agent outputs and performs three operations. First, deduplication: findings that refer to the same file and line are merged using a CWE plus file path plus line number composite key, with the highest-confidence source preserved. Second, cross-agent validation: findings independently reported by two or more agents receive an elevated confidence rating, reflecting the principle that agreement between deterministic and semantic analysis is a stronger signal than either alone. Third, composite risk score computation: R = w1 multiplied by S_sast plus w2 multiplied by S_sca plus w3 multiplied by S_secrets plus w4 multiplied by S_context, where S_context incorporates file sensitivity classification and the developer history factor derived from the longitudinal profile. Initial weights of 0.40, 0.25, 0.25, and 0.10 are set by expert elicitation and will be calibrated against ground-truth labels during Phase 4.

Stage 5 — Output and Gating. The batch review is posted to GitHub via the pull request reviews API endpoint with event type COMMENT, creating a single formal review thread containing all inline comments. Each comment includes the finding description, CWE identifier, CVSS score

estimate, and a structured fix prompt in the format required by the developer's detected stack. If the composite risk score exceeds the configured gating threshold, a GitHub Checks API status is posted as action_required with a summary of critical findings, blocking the pull request from merging under the repository's branch protection rules. All findings, scores, agent outputs, and metadata are persisted to PostgreSQL. Redis pub/sub notifies connected dashboard clients via Server-Sent Events.

3.5 Experimental Setup

Three experiments are planned, designed to test the four hypotheses and answer the four research questions stated in Section 3.1.

Experiment 1 — Ablation Study. The complete dataset of 1,200 labeled pull requests will be processed independently by seven pipeline configurations: (1) Semgrep only; (2) CodeQL only; (3) Trivy plus Bandit plus Gitleaks as the non-code-SAST scanner layer; (4) the full deterministic scanner suite without LLM agents; (5) LLM agents only without any scanner grounding; (6) Scanner output plus LLM agents, the CASARA configuration; (7) CASARA plus longitudinal developer profiling. Precision, recall, F1 score, false positive rate, and median end-to-end latency from webhook receipt to GitHub comment posting will be computed for each configuration against the ground-truth labels. Statistical significance of pairwise differences will be assessed using McNemar's test for paired binary classification data with Bonferroni correction for multiple comparisons.

Experiment 2 — Comparative Benchmarking. CASARA will be compared against three baseline systems on the same 1,200-pull-request dataset: GitHub Advanced Security with CodeQL and secret scanning enabled, SonarQube Community Edition with the default Java, Python, and JavaScript rule profiles, and PR-Agent deployed in self-hosted mode against the same repositories. Each baseline will be configured and deployed by the researcher against the same set of repositories under identical conditions, with all available features enabled. The same precision, recall, F1, false positive rate, and latency metrics will be computed and reported with 95 percent bootstrap confidence intervals.

Experiment 3 — User Study. The 40-participant balanced crossover user study will evaluate developer-facing outcomes. Primary outcome measures are Likert-scale ratings on four five-point dimensions: finding clarity (1 = incomprehensible to 5 = immediately clear), actionability (1 = no clear next step to 5 = could implement the fix immediately), false positive likelihood (1 = clearly correct to 5 = clearly incorrect), and fix prompt usefulness (1 = not useful to 5 = applied directly). Secondary outcomes are fix prompt acceptance rate, defined as the proportion of fix prompts applied without modification within a 72-hour window, and qualitative exit interview data coded using thematic analysis. Statistical analysis will use a linear mixed-effects model with participant

and pull request as random effects, condition and experience level as fixed effects, and Tukey post-hoc adjustment for pairwise comparisons.

3.6 Evaluation Metrics

The primary quantitative metrics are precision (true positives divided by the sum of true positives and false positives), recall (true positives divided by the sum of true positives and false negatives), F1 score as the harmonic mean of precision and recall, false positive rate (false positives divided by the sum of false positives and true negatives), and median end-to-end latency in seconds from webhook receipt to the first inline GitHub review comment. All metrics are computed at the finding level against CWE-annotated ground-truth labels.

Secondary metrics include composite risk score calibration assessed via Brier score and AUC-ROC for pull-request-level binary risk classification, fix prompt acceptance rate measured on a live deployment against partner repositories, developer Likert ratings across four dimensions in the user study, remediation time measured as the elapsed time from finding post to author-committed fix commit, and inter-rater agreement on the ground-truth dataset measured by Cohen's kappa.

CHAPTER 4: EXPECTED RESULTS

4.1 Quantitative Outcomes

The ablation study is expected to confirm H1: the full CASARA pipeline will achieve a statistically significantly higher F1 score than both scanner-only and LLM-only baselines. The basis for this projection is as follows. Independent evaluations by Chen et al. (2023) place Semgrep precision at approximately 85 percent on surface-level vulnerability patterns. Li et al. (2023) place GPT-4 security review precision at approximately 72 percent without scanner grounding. Under the assumption that grounding the LLM in scanner output eliminates false positives generated by the LLM on patterns already known to be clean by the scanner, and separately that the LLM correctly identifies semantic issues missed by the scanners, a combined F1 in the range of 0.83 to 0.87 is projected for the full pipeline. This projection is conservative; if the scanner-LLM grounding effect is stronger than assumed, performance could exceed this range.

The comparative benchmarking experiment is expected to show CASARA achieving parity or superiority with GitHub Advanced Security on standard SAST metrics, with a significant advantage on developer-facing metrics. GitHub Advanced Security provides scanner findings as status check annotations with no natural-language explanation and no fix guidance; CASARA's structured fix prompts address this gap directly. The most significant expected advantage over GitHub Advanced Security is in developer satisfaction ratings, where the difference in communication quality is anticipated to produce Likert rating differences of 1.5 to 2.0 scale points on clarity and actionability. Against SonarQube Community Edition, which similarly lacks LLM-based commentary and does not perform secrets detection or IaC scanning, CASARA is expected to show advantages across all measured dimensions. Against PR-Agent in LLM-only mode, the expected advantage is in false positive rate: scanner grounding should reduce the LLM false positive rate from the projected 28 to 35 percent range to below 15 percent, consistent with the mechanism hypothesized in H1.

For the composite risk score calibration, the target Brier score is below 0.15 and the target AUC-ROC is above 0.85 for pull-request-level binary risk classification. These targets are based on published calibration performance of risk scoring models in related domains. If the initial weight configuration does not achieve these targets, gradient-based weight optimization against the labeled dataset will be applied during Phase 4.

PROJECTED PERFORMANCE

Based on literature baselines and preliminary prototype evaluation: CASARA F1 target 0.83–0.87 (vs. scanner-only ~0.68–0.72 and LLM-only ~0.65–0.70 on the same dataset). False positive rate target below 15% (vs. LLM-only projected 28–35%). These projections are stated as falsifiable predictions, not guarantees.

4.2 Qualitative and Developer-Facing Outcomes

The user study is expected to confirm H3: CASARA findings will yield significantly higher developer satisfaction ratings than both raw scanner output and ungrounded LLM commentary. The clarity advantage is hypothesized to be most pronounced for junior developers, who are least equipped to interpret raw CWE references and CVSS scores without contextual explanation. The actionability advantage is hypothesized to be most pronounced for developers of all experience levels when comparing CASARA against scanner-only output, since fix prompts are entirely absent from raw scanner annotations.

These results are expected to confirm H2: fix prompt acceptance rates are expected to exceed 60 percent for scanner-grounded prompts compared to a projected 35 to 40 percent for ungrounded LLM prompts, based on the principle that developers are more willing to act on suggestions that can be verified against a formal analysis finding rather than relying solely on LLM assertion. Remediation time, measured as the elapsed period between finding post and author-committed fix, is expected to be 40 to 60 percent lower for CASARA conditions than for scanner-only conditions, reflecting the reduction in cognitive effort required to understand and implement a fix when a structured prompt is provided.

The longitudinal profiling analysis is expected to show a statistically significant reduction in repeat vulnerability category introduction when developers receive history-aware feedback, confirming H4. The expected mechanism is that LLM agents calibrated with an author's prior finding history produce commentary that references the recurrence explicitly, creating a more salient and memorable signal than generic feedback applied uniformly. The magnitude of this effect is difficult to project a priori; a medium effect size (Cohen's d = 0.5) is the minimum that would be considered practically significant for this research.

Beyond quantitative results, the research is expected to produce the following community-value artifacts: a labeled benchmark dataset of 1,200 pull requests released under CC BY 4.0 license; the CASARA framework released as open-source software under MIT license; a set of calibrated Semgrep rules contributed to the public rule registry; and the calibrated risk scoring model with published weight values and calibration methodology.

CHAPTER 5: TENTATIVE TIMELINE

The research programme spans 4 months from July to October 2026, organized in five sequential phases with clearly defined entry criteria, deliverables, and exit criteria at each milestone. The phases are designed to minimize dependency risk: each phase delivers a testable artifact that validates the assumptions of the subsequent phase before significant resources are committed to it.

Phase	Task Description	Duration	Key Deliverables	Date
Phase 1	Literature Review, Gap Analysis, and Baseline Benchmarking	3 weeks	SLR report, RQ/hypothesis set, baseline metrics	Jul 2026
Phase 2	System Architecture, Full Scanner Integration, and Prototype Development	5 weeks	Working CASARA prototype, labeled dataset v1	Jul–Aug 2026
Phase 3	Dataset Finalization, Live Deployment, and Experiment 1 Execution	4 weeks	Complete dataset, ablation results, user study cohort	Aug–Sep 2026
Phase 4	Comparative Benchmarking, User Study, and Statistical Analysis	4 weeks	All experimental results, statistical models	Sep–Oct 2026
Phase 5	Thesis Writing, Peer Review Submission, and Final Defense	3 weeks	Thesis draft, 1+ manuscript submitted, OSS release	Oct 2026

5.1 Phase Descriptions

Phase 1 (July 2026): Literature Review and Baseline Benchmarking. This phase conducts a focused literature review across the four domains identified in Chapter 2, producing a structured evidence base for the hypotheses. In parallel, the existing prototype is evaluated against the three comparison systems on a preliminary set of 50 manually labeled pull requests to establish a quantified baseline and identify the most impactful scanner integrations to prioritize in Phase 2. Exit criterion: literature review completed and baseline metrics documented.

Phase 2 (July–August 2026): Architecture Design and Prototype Development. Full integration of CodeQL, Trivy, Bandit, and Gitleaks into the Celery worker pipeline alongside the existing Semgrep and LLM agents. Design and implementation of the composite risk scoring

module, the longitudinal developer profile store, and the GitHub Checks API gating layer. Commencement of the benchmark dataset construction and annotation protocol. Exit criterion: complete CASARA prototype passing integration tests across all five scanner integrations, with risk scoring and gating operational.

Phase 3 (August–September 2026): Dataset Completion and Experiment 1. Finalization of the labeled dataset with Cohen's kappa verification. Live deployment of CASARA on two to three partner open-source repositories for ecological validity assessment. Execution of Experiment 1 across all seven pipeline configurations. Recruitment and scheduling of user study participants. Exit criterion: Experiment 1 complete, user study cohort confirmed.

Phase 4 (September–October 2026): Comparative Benchmarking and User Study. Execution of Experiment 2 with all three baseline systems on the complete dataset. Execution of Experiment 3 with all 40 participants. Comprehensive data analysis including statistical testing, effect size estimation, and visualization. Calibration of composite risk score weights against ground-truth labels. Exit criterion: all three experiments analyzed with findings documented.

Phase 5 (October 2026): Writing and Dissemination. Research report drafting and supervisor review. Preparation of a target journal or conference manuscript. Coordinated open-source release of the framework, dataset, and Semgrep rules. Exit criterion: report submitted for examination, at least one manuscript under review.

CHAPTER 6: EXPECTED CONTRIBUTIONS AND SIGNIFICANCE

This research is expected to produce contributions at three distinct levels of abstraction: theoretical, establishing a new design principle for hybrid analysis systems; empirical, producing validated artifacts that support future comparative research; and practical, delivering a production-ready framework that addresses an unmet need in the daily practice of software engineering teams.

Theoretical Contribution. The primary theoretical contribution is a formal characterization of the complementarity between deterministic program analysis and LLM-based semantic reasoning in the specific context of code security review. The claim that grounding LLM agents in structured scanner output reduces false positive rates while preserving or improving recall has not previously been formally stated as a research hypothesis, operationalized into a measurable experimental design, or subjected to empirical testing. If confirmed at the anticipated effect sizes, this result establishes a design principle with broad applicability: any task that benefits from both formal rule-based analysis and semantic language understanding can potentially leverage a similar hybrid grounding architecture. The CASARA research is intended to provide the first rigorous empirical evidence for or against this design principle.

Empirical Contribution. The research delivers two open empirical artifacts that are intended to outlast the specific tools integrated in this iteration of CASARA. The labeled benchmark dataset of 1,200 real-world pull requests with CWE-annotated, multi-tool-annotated ground-truth labels is designed to become a standard reference corpus for evaluating future automated code review tools, analogous to the role that Defects4J has played in automated program repair research. The calibrated composite risk scoring model, with its weight values, calibration methodology, and performance characterization on the benchmark, provides a reference implementation that organizations can adopt or adapt for their own merge policy frameworks.

Practical Contribution. The CASARA framework addresses a gap that affects every software engineering team using pull requests as a quality gate. The current industry norm requires teams to choose between coverage, provided by scanner annotations that most developers cannot interpret, and usability, provided by LLM review assistants that lack formal analytical grounding. CASARA demonstrates that this choice is a false dichotomy. The framework reduces the specialist security knowledge required to act on vulnerability findings, which has direct implications for the majority of development teams that lack dedicated application security resources. By generating developer-readable, context-aware fix prompts grounded in formal analysis, CASARA enables developers at all experience levels to engage with security findings effectively, shifting the cost of vulnerability remediation earlier in the development lifecycle where it is lowest.

Significance for the Field. The longitudinal developer profiling contribution represents a conceptual advance in the framing of automated code review. By treating the review pipeline as a mechanism for personalized developer education rather than merely a quality enforcement gate, CASARA aligns automated security tooling with the goal of reducing the rate at which vulnerabilities are introduced in the first place. This shift from reactive detection to proactive capability development is consistent with the broader direction of the software engineering field toward developer-centric security practice, sometimes described as shifting security left. The research provides the first empirical evidence on whether automated personalized security feedback, delivered at scale through a pull-request pipeline, can measurably improve developer vulnerability patterns over time.

Publication targets include IEEE Transactions on Software Engineering, the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), and IEEE Security and Privacy Workshops. The open-source framework, dataset, and trained artifacts will be released on GitHub under permissive licenses coordinated with the submission timeline to maximize reproducibility and community adoption.

REFERENCES

[1] P. Avgustinov, O. de Moor, M. P. Jones, and M. Verbaere, "QL: Object-Oriented Queries on Relational Data," in Proc. 30th European Conference on Object-Oriented Programming (ECOOP), ser. Leibniz International Proceedings in Informatics, vol. 56, 2016, pp. 2:1–2:26. DOI: 10.4230/LIPIcs.ECOOP.2016.2.

[2] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software," in Proc. 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE), New York, NY, USA: ACM, 2021, pp. 30–39. DOI: 10.1145/3475960.3475985.

[3] D. Brotman, L. Donahue, and I. Musselman, "Semgrep: A Lightweight Generic Static Analysis Tool," in Proc. 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), New York, NY, USA: IEEE, 2021, pp. 281–282. DOI: 10.1109/ICSE-Companion52605.2021.00115.

[4] L. Chen, J. Han, Y. Zhang, and X. Liu, "A Comparative Evaluation of Static Analysis Security Testing Tools for Python and JavaScript Applications," in Proc. IEEE International Conference on Software Quality, Reliability, and Security (QRS), Chiang Mai, Thailand: IEEE, 2023, pp. 45–54. DOI: 10.1109/QRS60937.2023.00016.

[5] GitHub, "CodeQL Documentation: About Code Scanning with CodeQL," GitHub Advanced Security, San Francisco, CA, USA, 2024. [Online]. Available: https://docs.github.com/en/code-security/code-scanning

[6] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, Z. Wang, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," in Proc. 12th International Conference on Learning Representations (ICLR), 2024. arXiv:2308.00352.

[7] IBM Security, "Cost of a Data Breach Report 2023," IBM Corp., Armonk, NY, USA, Tech. Rep., 2023. [Online]. Available: https://www.ibm.com/reports/data-breach

[8] Aqua Security, "Trivy: A Comprehensive Vulnerability Scanner for Containers and Other Artifacts," Aqua Security Software Ltd., Tel Aviv, Israel, 2020. [Online]. Available: https://github.com/aquasecurity/trivy

[9] G. A. Kildall, "A Unified Approach to Global Program Optimization," in Proc. 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL), New York, NY, USA: ACM, 1973, pp. 194–206. DOI: 10.1145/512927.512945.

[10] C. Kulkarni, K. P. Wei, H. Le, D. Chia, K. Papadopoulos, J. Cheng, D. Koller, and S. R. Klemmer, "Peer and Self Assessment in Massive Online Classes," ACM Transactions on Computer-Human Interaction, vol. 20, no. 6, pp. 33:1–33:31, Dec. 2013. DOI: 10.1145/2505057.

[11] Y. Li, Z. Wang, and K. Shen, "Evaluating Large Language Models for Security-Oriented Code Review: A Controlled Experiment," in Proc. IEEE International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy: IEEE, 2023, pp. 112–122. DOI: 10.1109/ISSRE59848.2023.00019.

[12] V. B. Livshits and M. S. Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis," in Proc. 14th USENIX Security Symposium, Baltimore, MD, USA: USENIX, 2005, pp. 271–286.

[13] M. Meli, M. R. McNiece, and B. Reaves, "How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories," in Proc. Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA: Internet Society, 2019. DOI: 10.14722/ndss.2019.23418.

[14] M. Tufano, N. Deng, R. Drain, D. Svyatkovskiy, S. K. Dey, M. Sundaresan, and N. Nagappan, "Using Pre-Trained Models to Boost Code Review Automation," in Proc. 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA: ACM/IEEE, 2022, pp. 2291–2302. DOI: 10.1145/3510003.3510621.

[15] C. Pornprasit and C. Tantithamthavorn, "JITLine: A Simpler, Better, Faster, Finer-Grained Just-In-Time Defect Prediction," in Proc. 18th International Conference on Mining Software Repositories (MSR), Madrid, Spain: IEEE, 2021, pp. 369–379. DOI: 10.1109/MSR52588.2021.00052.

[16] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Communicative Agents for Software Development," in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Bangkok, Thailand: ACL, 2024. arXiv:2307.07924.

[17] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An Analysis of the Automatic Bug Fixing Performance of ChatGPT," in Proc. IEEE/ACM International Workshop on Automated Program

Repair (APR), Melbourne, Australia: IEEE, 2023, pp. 23–29. DOI: 10.1109/APR59189.2023.00010.

[18] Sonatype, "2023 State of the Software Supply Chain Report," Sonatype Inc., Fulton, MD, USA, Tech. Rep., 2023. [Online]. Available: https://www.sonatype.com/state-of-the-software-supply-chain

[19] Y. Zeng, Z. Cao, B. Qin, and L. Liu, "An Empirical Study of Deep Learning Models for Vulnerability Detection," in Proc. 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA: ACM/IEEE, 2022, pp. 2094–2105. DOI: 10.1145/3510003.3510283.

[20] OpenAI, "GPT-4 Technical Report," OpenAI, San Francisco, CA, USA, Tech. Rep. arXiv:2303.08774, Mar. 2023.

[21] Anthropic, "Claude 3 Model Card," Anthropic, San Francisco, CA, USA, Tech. Rep., Mar. 2024. [Online]. Available: https://www.anthropic.com/index/claude-3-family

[22] Konvu, "Semgrep vs. CodeQL: Technical Comparison for Security Teams," Konvu Inc., 2024. [Online]. Available: https://konvu.com/compare/semgrep-vs-codeql

APPENDICES

Appendix A — System Architecture Overview

The five-layer CASARA pipeline is described component by component below. Each component is independently deployable and communicates with adjacent components through well-defined interfaces, enabling substitution or upgrade of individual components without requiring full system redeployment.

Ingress Layer. The FastAPI endpoint at /webhooks/github validates HMAC-SHA256 signatures, checks Redis for the X-GitHub-Delivery idempotency key with a 24-hour TTL, and enqueues Celery tasks within the GitHub webhook timeout budget. The endpoint is the sole entry point for all GitHub events; event routing to different task types (pull-request review versus issue comment or push event handling) occurs within the endpoint handler.

Scanner Layer. Five scanner processes are invoked as parallel asyncio subprocesses from within the Celery worker. Each process writes its output to a temporary file in a shared volume; the worker collects all outputs after a configurable timeout (default 120 seconds) and normalizes them. Scanners that exceed their timeout are logged and their partial output is used; the pipeline is designed to degrade gracefully under scanner failure without blocking the LLM analysis stage.

LLM Agent Layer. Five agent classes inherit from a shared BaseAgent that handles Anthropic SDK initialization, token budget management, structured output validation, and cost tracking. Each agent class overrides the system_prompt and output_schema properties. Agents are invoked via asyncio.gather with individual token limits enforced per agent to bound total pipeline cost.

Aggregation Layer. The Aggregator is implemented as a sixth LLM agent that receives the five agent outputs as a structured JSON array. Its system prompt instructs it to deduplicate, validate, rank, and compute the composite risk score. The Aggregator's output is a single structured JSON document conforming to the final review schema.

Output Layer. GitHub inline comments are posted via a single batch review API call to minimize API quota consumption and produce a single coherent review thread rather than scattered individual comments. The Checks API gating status is a separate API call posted immediately after the review, allowing PR authors to see the risk decision alongside the review findings.

Appendix B — Composite Risk Score Formula

The composite risk score R for a pull request is defined as:

$$R = w_1 \cdot S\_sast + w_2 \cdot S\_sca + w_3 \cdot S\_secrets + w_4 \cdot S\_context$$

S_sast is the normalized SAST severity score: the sum of CVSS estimates across all confirmed SAST findings, normalized to the range [0, 10] by dividing by the maximum achievable score for the number of files changed.

S_sca is the SCA risk score: the maximum CVSS score across all dependency vulnerabilities introduced or upgraded in the pull request, taken from the NVD record for the corresponding CVE.

S_secrets is a binary-weighted secrets score: 10 if any confirmed secret with verified validity is detected, 5 if any unverified high-probability secret is detected, and 0 otherwise.

S_context is the context sensitivity and history score, normalized to [0, 10]: S_context = ((sensitivity_factor × history_factor) − 1.0) × 5, where sensitivity_factor ∈ {1.0, 1.5, 2.0} reflects file security classification (1.0 for general code; 1.5 for authentication, session, or authorization modules; 2.0 for cryptography, payment processing, or key management code), and history_factor ∈ [1.0, 1.5] is elevated proportionally to the count of prior findings in the same CWE category by the pull request author within the preceding 90 days. This normalization ensures S_context ∈ [0, 10], consistent with the other component scores.

Initial weights w1 = 0.40, w2 = 0.25, w3 = 0.25, w4 = 0.10 are set by expert elicitation. These will be optimized against the labeled dataset using gradient descent on the Brier score loss during Phase 4 of the research.

Appendix C — Sample Agent System Prompt (Security Agent)

The following excerpt illustrates the Security Agent system prompt structure used in the deployed CASARA prototype. Prompt engineering choices are annotated in parentheses for transparency.

```
You are a senior application security engineer conducting a formal security
review of a pull request. You will be provided with: (1) the complete diff of
changed files; (2) structured findings in JSON format from Semgrep, CodeQL,
Trivy, Bandit, and Gitleaks — treat these as verified signals; (3) repository
language, framework, and dependency context; (4) the author's historical
vulnerability profile. Your task is to produce a security analysis conforming
exactly to the following JSON schema: { issues: [ { cwe_id: string, severity:
```

```
'critical'|'warning'|'info', cvss_estimate: number, file: string, line: number, description: string, exploitability: string, fix_prompt: string, confidence: 'HIGH'|'MEDIUM'|'LOW' } ] }. Ground every HIGH confidence finding in a scanner signal where available. Mark findings not supported by scanner output as MEDIUM or LOW confidence. For each finding, write the description and exploitability in plain language accessible to a developer without a security background. The fix_prompt field must be a complete, actionable instruction including the specific change required.
```

Appendix D — Algorithm Pseudocode

The following pseudocode describes the Aggregation and Risk Scoring stage (Stage 4) of the CASARA pipeline at a level of detail sufficient for independent implementation.

```
FUNCTION aggregate_and_score(agent_outputs[], developer_history, changed_files[]):
all_findings = FLATTEN(agent_outputs)    // Deduplication by composite key
seen_keys = {}   deduped = []   FOR finding IN all_findings:     key = (finding.cwe_id, finding.file, finding.line)     IF key IN seen_keys:
seen_keys[key].confidence = MAX(seen_keys[key].confidence, finding.confidence)
seen_keys[key].sources.APPEND(finding.source_agent)     ELSE:       seen_keys[key] = finding       deduped.APPEND(finding)    // Cross-agent validation boost   FOR finding IN deduped:     IF LENGTH(finding.sources) >= 2:       finding.confidence = 'HIGH'    // Compute component scores   S_sast     = NORMALIZE(SUM(f.cvss_estimate FOR f IN deduped IF f.source IN SAST_TOOLS))   S_sca      = MAX(f.cvss_estimate FOR f IN deduped IF f.source == 'TRIVY_SCA', default=0)   S_secrets  = 10 IF ANY(f.source=='GITLEAKS' AND f.verified FOR f IN deduped) ELSE                5 IF ANY(f.source=='GITLEAKS' FOR f IN deduped) ELSE 0   sensitivity = CLASSIFY_FILES(changed_files)   // 1.0 | 1.5 | 2.0   history_f   = COMPUTE_HISTORY_FACTOR(developer_history)  // 1.0–1.5   S_context   = (sensitivity * history_f - 1.0) * 5.0  // normalized to [0,10]    R = 0.40*S_sast + 0.25*S_sca + 0.25*S_secrets + 0.10*S_context    RETURN { findings: SORT_BY_SEVERITY(deduped), risk_score: R, gate: R >= THRESHOLD }
```